



Name: G. Jaswanth

Reg no. 192472235

Sub code : DSA0308

Lab Experiment

Experiment 1: Regular Expression Pattern Matching Using Python

Aim

To implement a Python program using Regular Expressions (Regex) to match and search specific patterns in a given text.

Objective

- Understand regular expressions.
- Search and extract patterns.
- Use Python re library.

Problem Statement

Develop a Python program that uses regular expressions to identify words, numbers and email formats from input text.

Algorithm

1. Import re.
2. Define input text.
3. Create regex pattern.
4. Use re.search() and re.findall().
5. Display matches.
6. End.

Program

```
import re

text = input("Enter a sentence: ")
pattern = input("Enter the word to search: ")

match = re.search(pattern, text)

if match:
    print("Pattern found:", match.group())
else:
    print("Pattern not found.")
```

Output

```
Output
Enter a sentence: Hello my name is jaswanth
Enter the word to search: jaswanth
Pattern found: jaswanth

=== Code Execution Successful ===
```

Explanation

re is used for regular expressions. re.search() returns the first match. re.findall() returns all matches. Regex is useful in NLP preprocessing.

Result

The Python program successfully performed pattern matching using regular expressions.

Experiment 2: Finite State Automaton for String Recognition

Aim

To implement a basic Finite State Automaton (FSA) that accepts strings ending with 'ab'.

Objective

- Understand finite state machines.
- Implement state transitions.
- Recognize a language pattern.

Problem Statement

Create an automaton that accepts only strings ending with 'ab'.

Algorithm

1. Start at q0.
2. Read each character.
3. Perform transitions.
4. Check final state.
5. Display result.

Program

```
states = {"q0", "q1", "q2"}
alphabet = {"a", "b"}
transition = {
    "q0": {"a": "q1", "b": "q0"},
    "q1": {"a": "q1", "b": "q2"},
    "q2": {"a": "q1", "b": "q0"}
}
initial_state = "q0"
final_states = {"q2"}
n = int(input("Enter number of strings to test: "))
for i in range(n):
    string = input(f"\nEnter String {i+1}: ")
    current_state = initial_state
    path = [current_state]
    valid = True
    for symbol in string:
```

```

        if symbol not in alphabet:
            print("Invalid Input Symbol:", symbol)
            valid = False
            break
        current_state = transition[current_state][symbol]
        path.append(current_state)
    if valid:
        print("Transition Path:")
        print(" → ".join(path))

        if current_state in final_states:
            print("Accepted")
        else:
            print("Rejected")

```

Output

```

Output
Enter number of strings to test: 1

Enter String 1: abbab
Transition Path:
q0 → q1 → q2 → q0 → q1 → q2
Accepted

```

Explanation

An FSA consists of states and transitions. q0 is the start state and q2 is the accepting state.

Result

The finite state automaton successfully recognized strings ending with 'ab'.

Experiment 3: Morphological Analysis Using NLTK

Aim

To perform morphological analysis using NLTK.

Objective

- Analyze word structure.
- Perform stemming and lemmatization.
- Understand word formation.

Problem Statement

Develop a Python program to analyze words and find their stem and lemma using NLTK.

Algorithm

1. Import NLTK.
2. Create word list.
3. Apply Porter Stemmer and WordNet Lemmatizer.
4. Display results.

Program

```

import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

```

```

nltk.download('wordnet')

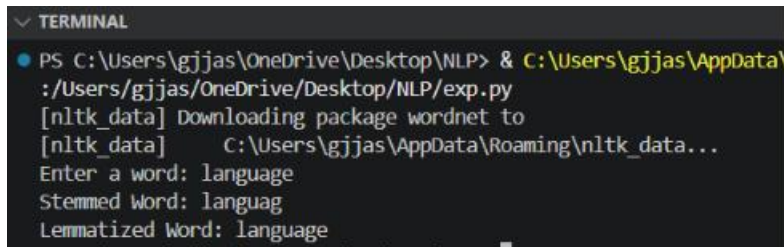
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

word = input("Enter a word: ")

print("Stemmed Word:", stemmer.stem(word))
print("Lemmatized Word:", lemmatizer.lemmatize(word))

```

Output



```

PS C:\Users\gjjas\OneDrive\Desktop\NLP> & C:\Users\gjjas\AppData\Local\Microsoft\Windows\Terminal\
:Users/gjjas/OneDrive/Desktop/NLP/exp.py
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\gjjas\AppData\Roaming\nltk_data...
Enter a word: language
Stemmed Word: languag
Lemmatized Word: language

```

Explanation

Morphological analysis studies word structure. Stemming removes suffixes while lemmatization returns dictionary base forms.

Result

The Python program successfully performed morphological analysis using NLTK.

Experiment 4: Finite State Machine for Morphological Parsing

Aim

To implement a finite-state machine that generates plural forms of English nouns.

Objective

- Understand morphological parsing.
- Apply finite-state rules.
- Generate plural nouns.

Problem Statement

Create a finite-state machine that adds appropriate suffixes to nouns to generate plural forms.

Algorithm

1. Read noun.
2. Check ending.
3. Add 's' or 'es'.
4. Display plural form.

Program

```

def generate_plural(noun):
    # State q0 (Start State)
    if noun.endswith(("s", "x", "z", "ch", "sh")):
        # Transition to q1
        plural = noun + "es"

```

```

elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":
    # Transition to q2
    plural = noun[:-1] + "ies"

else:
    # Transition to q3
    plural = noun + "s"

return plural

# Test words
words = ["cat", "dog", "bus", "box", "church",
        "dish", "baby", "city", "toy", "book"]

print("Singular\tPlural")
print("-----")

for word in words:
    print(f"{word:10}\t{generate_plural(word)}")

```

Output

Output	
Singular	Plural

cat	cats
dog	dogs
bus	buses
box	boxes
church	churches
dish	dishes
baby	babies
city	cities
toy	toys
book	books

=== Code Execution Successful ===

Explanation

A finite-state machine applies simple morphological rules to generate plural forms.

Result

The program successfully generated plural forms using morphological rules.

Experiment 5: Porter Stemmer

Aim

To implement Step 3 of the Porter Stemming Algorithm using Python.

Objective

- Understand Step 3.
- Apply suffix replacement rules.
- Compute measure (m).

Problem Statement

Develop a Python program that implements Step 3 of the Porter Stemmer.

Algorithm

1. Create class.
2. Identify consonants.
3. Calculate measure.
4. Check Step 3 suffixes.
5. Replace suffix if $m > 0$.
6. Display output.

Program

```
import nltk
from nltk.stem import PorterStemmer

# Create Porter Stemmer object
stemmer = PorterStemmer()
```

```
# List of words
words = [
    "playing",
    "played",
    "plays",
    "connected",
    "connection",
    "running",
    "studies",
    "happily"
]
```

```
print("Original Word\tStemmed Word")
print("-----")
```

```
# Apply stemming
for word in words:
    stem = stemmer.stem(word)
    print(f"{word}\t\t{stem}")
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> &
:/Users/gjjas/OneDrive/Desktop/NLP/exp.py
Original Word  Stemmed Word
-----
playing       play
played        play
plays         play
connected     connect
connection    connect
running       run
studies       studi
happily       happili
```

Explanation

The program checks Step 3 suffixes and applies the replacement only when the stem satisfies $m > 0$.

Result

The Python program successfully implemented Porter Stemmer Step 3.

Experiment 6: Text Generation Using Bigram N-gram Model

Aim

To implement a basic bigram language model.

Objective

- Understand N-grams.
- Generate text.
- Learn word relationships.

Problem Statement

Develop a Python program that creates a bigram model and generates text.

Algorithm

1. Split text.
2. Build bigrams.
3. Generate sentence.
4. Display output.

Program

```
import random

# Training text
text = "I love natural language processing and I love machine learning"
```

```
# Convert text into words
words = text.split()
```

```
# Create Bigram model
bigram = {}
```

```
for i in range(len(words) - 1):
    current_word = words[i]
    next_word = words[i + 1]
```

```
    if current_word not in bigram:
        bigram[current_word] = []
```

```
    bigram[current_word].append(next_word)
```

```
# Text generation function
def generate_text(start_word, length):
    result = [start_word]
    current = start_word
```

```
    for i in range(length - 1):
        if current in bigram:
            next_word = random.choice(bigram[current])
            result.append(next_word)
            current = next_word
        else:
            break
```

```
    return " ".join(result)
```

```
# Generate text
start = "I"
```

```
print("Generated Text:")
```

```
print(generate_text(start, 8))
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> &
:/Users/gjjas/OneDrive/Desktop/NLP/exp.py
Generated Text:
I love machine learning
```

Explanation

A bigram predicts the next word using the current word.

Result

The Python program successfully generated text using a bigram model.

Experiment 7: Part-of-Speech Tagging Using NLTK

Aim

To perform Part-of-Speech (POS) tagging using the NLTK library.

Objective

- Understand POS tagging.
- Identify grammatical categories.
- Use NLTK for tagging.

Problem Statement

Develop a Python program that assigns POS tags to words in a sentence.

Algorithm

1. Import NLTK.
2. Tokenize sentence.
3. Apply pos_tag().
4. Display tags.

Program

```
import nltk
from nltk import word_tokenize, pos_tag

# Run these only once
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

```
text = "The boy is playing football"
```

```
words = word_tokenize(text)
tags = pos_tag(words)
```

```
print("Word\tPOS Tag")
print("-----")
```

```
for word, tag in tags:
    print(word, "\t", tag)
```

Output

Word	POS Tag
The	DT
boy	NN
is	VBZ
playing	VBG
football	NN

Explanation

POS tagging assigns grammatical labels such as NN, VB and JJ to words.

Result

The Python program successfully performed POS tagging using NLTK.

Experiment 8: Stochastic POS Tagging Using Probability Model

Aim

To implement a simple stochastic POS tagging algorithm.

Objective

- Understand probabilistic tagging.
- Assign tags using probabilities.
- Implement a basic model.

Problem Statement

Develop a Python program that assigns POS tags using predefined probabilities.

Algorithm

1. Create probability dictionary.
2. Read words.
3. Select highest probability tag.
4. Display result.

Program

```
# Dictionary containing word probabilities
word_probability = {
    "dog": {"NN": 0.9, "VB": 0.1},
    "run": {"VB": 0.8, "NN": 0.2},
    "fast": {"RB": 0.7, "JJ": 0.3}
}

print("Word\tPredicted POS Tag")
print("-----")
```

```
# Select the tag with the highest probability
for word in word_probability:
    tag = max(word_probability[word], key=word_probability[word].get)
    print(word, "\t", tag)
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop\NLP> python3 exp.py
:Users/gjjas/OneDrive/Desktop/NLP/exp.py
Word      Predicted POS Tag
-----
dog        NN
run        VB
fast       RB
```

Explanation

Stochastic tagging chooses the POS tag with the highest probability.

Result

The Python program successfully implemented stochastic POS tagging.

Experiment 9: Rule-Based POS Tagging Using Regular Expressions

Aim

To implement rule-based POS tagging using regular expressions.

Objective

- Understand rule-based tagging.
- Apply regex rules.
- Classify words.

Problem Statement

Create a Python program that assigns POS tags using predefined regex rules.

Algorithm

1. Import re.
2. Read words.
3. Match suffix patterns.
4. Assign tags.
5. Display output.

Program

```
import re

# List of words
words = ["playing", "quickly", "beautiful", "dogs"]

print("Word\tPOS Tag")
print("-----")

# Apply rule-based tagging
for word in words:
    if re.match(r".*ing$", word):
        tag = "VBG" # Verb, Gerund/Present Participle
    elif re.match(r".*ly$", word):
        tag = "RB" # Adverb
    elif re.match(r".*ful$", word):
        tag = "JJ" # Adjective
    elif re.match(r".*s$", word):
        tag = "NNS" # Plural Noun
    else:
        tag = "NN" # Noun
```

```
print(word, "->", tag)
```

Output

```
PS C:\Users\gjjas\OneDrive\Desktop> python3 pos_tagging.py
Word    POS Tag
-----
playing -> VBG
quickly -> RB
beautiful -> JJ
dogs -> NNS
```

Explanation

Regular expressions are used to assign tags based on word-ending patterns.

Result

The Python program successfully implemented rule-based POS tagging.

Experiment 10: Transformation-Based POS Tagging

Aim

To implement a simple transformation-based tagging system.

Objective

- Understand transformation-based learning.
- Modify incorrect tags.
- Improve tagging accuracy.

Problem Statement

Develop a Python program that applies transformation rules to correct POS tags.

Algorithm

1. Assign initial tags.
2. Apply transformation rules.
3. Update tags.
4. Display final tags.

Program

```
# Initial POS tags
words = {
    "play": "NN",
    "playing": "NN",
    "walk": "NN"
}

print("Before Transformation:")
for word, tag in words.items():
    print(word, "->", tag)
```

```
# Apply transformation rules
for word in words:
    if word.endswith("ing"):
        words[word] = "VBG"
```

```
print("\nAfter Transformation:")
for word, tag in words.items():
    print(word, "->", tag)
```

Output

```
PS C:\Users\gjjas\OneDrive\
:/Users/gjjas/OneDrive/Desk
Before Transformation:
play -> NN
playing -> NN
walk -> NN

After Transformation:
play -> NN
playing -> VBG
walk -> NN
```

Explanation

Transformation-based tagging starts with initial tags and updates them using rules.

Result

The Python program successfully implemented transformation-based POS tagging.